Touchline · Build Spec

Oct 18, 2026 · @push

Overview

Touchline (working name) is a multi-agent system that turns a live synthetic match feed into explainable, personalized on-screen intelligence, and its one promise is trust: every number on screen traces to an event, and no call is explained before the officials decide.

- **Who it serves:** broadcasters and streaming apps (timed overlay feed), stadium screens, fans in any language, analysts, and match officials' post-match briefings.
- **Primary category:** Best Multi-Agent System. Secondary fit: Best Enterprise Solution, through the grounding validator, decision state machine and producer console.
- **Deadline:** submissions close Oct 27, 2026, 11:59 pm Pacific (Oct 28, 2:59 am Indianapolis). Plan to submit by the afternoon of Oct 27.
- **Starting point:** the existing repo already has the simulator, offside engine, replay builder, replay player and 30 passing tests.

Done means:

1. A full synthetic match streams through Azure and drives live overlays end to end.
2. Casual and Analyst viewers see genuinely different screens, with club, player focus, metric pin and language applied.
3. Every call (VAR check, foul and card, goal) gets a replay explained for four audiences.
4. The eval harness reports latency, grounding pass rate, fallback rate and the premature-reveal test, and all pass.
5. A public demo link, public repo, README and a sub-two-minute video are submitted.

Requirements

The IDs below are what the eval harness and tests check against; each maps to a stage of the brief's five-stage pipeline.

Functional

ID	Stage	Requirement	Verified by
F1	Ingest	Simulator streams a full 90-minute match (passes, carries, shots, tackles, interceptions, pressure, fouls, cards, VAR events, metadata) into Event Hubs in scaled real time	Stream smoke test, event count per match
F2	Ingest	Scenarios are seeded and reproducible; at least three scripted scenarios (default match, late comeback, red card)	Determinism test
F3	Interpret	Stats engine derives player speed and distance, pass length, pass difficulty, ball and shot speed, rolling possession, control and pressure from positions	Unit tests with fixed frames
F4	Interpret	Offside check measures playable body parts only (Law 11), finds the kick point from a 100 fps ball track	Existing offside tests
F5	Interpret	Decision state machine holds on-field call, VAR outcome and final decision for every incident; no output may say "check complete" without the final decision	State machine tests
F6	Explain	Moment detector scores events and selects what gets airtime	Golden-match fixture
F7	Explain	Explainer and Decision Desk agents write why a moment matters, citing event IDs; reason codes explain slow checks	Grounding validator
F8	Render	Timed, machine-readable overlay feed (JSON) drives a web renderer: scorebug, tags, chips, momentum, VAR badge, lines, caption band, recap	Contract tests

ID	Stage	Requirement	Verified by
F9	Render	Call replays animate the build-up of every call with annotated moments	Existing replay tests
F10	Render	Producer console can approve, hold or kill any overlay; every aired item is written to an audit log	Console integration test
F11	Personalize	Casual and Analyst modes show different overlay sets, not reworded text	Snapshot test per mode
F12	Personalize	Favorite club reframes wording; player focus follows one player all match; metric pin keeps one chosen stat on screen	Snapshot tests
F13	Personalize	English and Spanish at minimum; referee announcement translated to subtitles	Translation fixture
F14	Recap	Half-time and full-time recaps from the stored moment log	Grounding validator

Non-functional

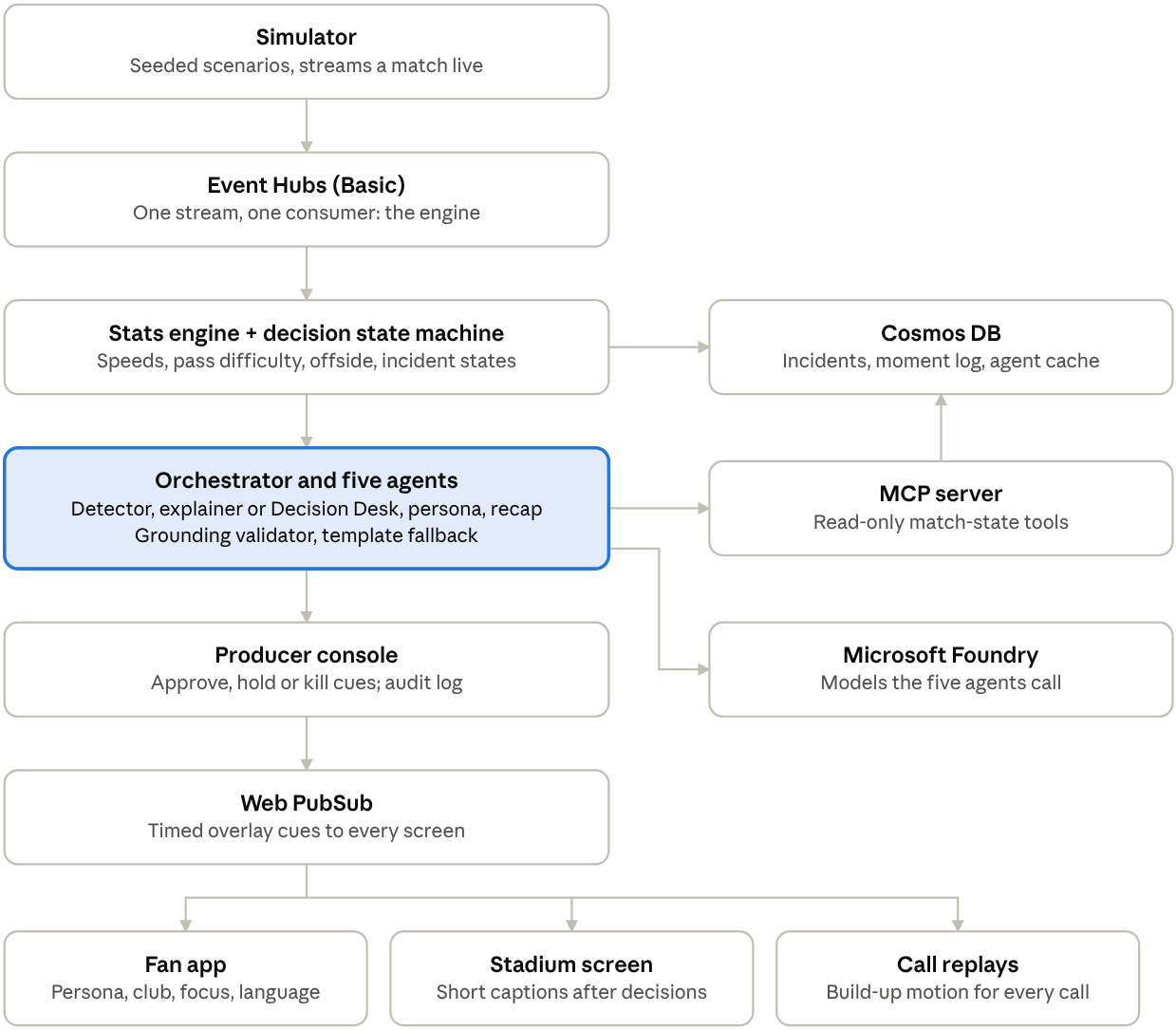
ID	Requirement	Target
N1	Event to overlay latency, deterministic overlays	p95 under 500 ms
N2	Event to overlay latency, agent-written text	p95 under 2.5 s
N3	Grounding: every number and event ID in agent text exists in its fact packet	100% of aired text; first-try pass rate reported
N4	Premature reveal: no margin, measurement or verdict before the decision event	0 violations, enforced by test

ID	Requirement	Target
N5	Failure recovery: an agent timeout or validator rejection falls back to a template	Overlay never blank; fallback rate reported
N6	Cost	Whole build and judging period inside the $100 Azure for Students credit
N7	Availability	Demo link up from Oct 27 to Nov 10 with a cached match that needs no live model calls
N8	Content	Fictional clubs and players only; no Premier League marks, no betting language, no copyrighted music

Architecture

The pipeline runs top to bottom: deterministic code computes every fact, the agents only turn facts into words, and nothing reaches a screen without passing the validator and the producer console.

Facts flow down from the engine; agents only add words



Every stage emits OpenTelemetry spans to Application Insights.

system architecture · 13 components

The engine is the only writer of match facts; the agents read them through the MCP server and call models in Foundry, and the orchestrator is the only path from agents to screens.

Runtime topology:

- **Container Apps (4 apps, scale to zero):** engine, MCP server, orchestrator, console API. The simulator runs as a Container Apps job, started per match.
- **Static Web Apps:** fan app, stadium view, call replays and console UI, plus the cached demo cue file.

- **Shared state:** Cosmos DB holds incidents, the moment log, the audit log and the agent output cache; the engine keeps live rolling metrics in memory and snapshots them to Cosmos each minute of match time.
- **Identity:** one user-assigned managed identity for all container apps, with data-plane roles on Event Hubs, Cosmos, Web PubSub and the Foundry project.

Data contracts

Four JSON contracts hold the system together; define them as Pydantic models in `contracts/` and generate TypeScript types for the frontend from the same models, so the two sides cannot drift.

1. Match event (simulator → Event Hubs)

Already implemented in `sim/simulator.py`. Every event carries all 22 positions, so speeds are derived, never stated.

```
{
  "event_id": "m01-e2104", "match_id": "m01", "period": 2, "clock": 4679.4,
  "type": "pass", "team": "NBR", "player": "NBR-8", "receiver": "NBR-9",
  "outcome": "complete", "start": [60.0, 40.0], "end": [84.91, 26.0],
"ball_speed_kmh": 79.2,
  "positions": { "NBR-9": [84.91, 26.0], "ASH-4": [85.0, 30.0] },
  "frame": { "clock": 4679.4, "body": { "NBR-9": { "foot_l": [85.21, 25.9],
"arm": [85.36, 26.25] } } },
  "ball_track": [[4679.25, 59.99, 40.0]]
}
```

`frame` and `ball_track` appear only on passes that may lead to an offside check. Event types: `kickoff, pass, carry, shot, tackle, interception, pressure, foul, card, goal, var_check, var_decision, period_end`.

2. Incident state (stats engine → agents, overlays)

One record per incident, owned by the decision state machine. Agents read it; only the engine writes it.

```
{
  "incident_id": "inc-m01-03", "kind": "offside", "goal_event": "m01-e2106",
  "state": "decided", "on_field": "goal", "var_outcome": "offside", "final":
"no_goal",
  "reason_codes": ["tight_margin"], "opened_at": 4685.0, "decided_at":
4720.0,
  "facts": { "margin_m": 0.31, "kick_clock": 4679.4, "attacker": "NBR-9",
"defender": "ASH-4" }
}
```

States run `open → reviewing → decided → restarted`. Reason codes: `tight_margin`, `subjective_interference`, `obstructed_view`, `multiple_phases`, `earlier_offence`. Facts unlock for overlays only in `decided`.

3. Overlay cue (orchestrator → producer console → renderer)

```
{
  "cue_id": "cue-0481", "at": 4720.0, "ttl_s": 8, "kind": "var_decision",
  "audience": { "persona": "casual", "club": "NBR", "focus": "NBR-9", "lang":
"es", "pin": "sprint_speed" },
  "layers": ["scorebug", "var_badge", "offside_lines", "band"],
  "band": { "tag": "No gol", "text": "Fuera de juego · Okafor 31 cm
adelantado en el pase" },
  "graphics": { "offside_lines": { "attacker_x": 84.91, "defender_x": 84.6 }
},
  "sources": ["m01-e2104", "m01-e2106", "inc-m01-03"],
  "origin": "agent", "validated": true, "approval": "auto"
}
```

`origin` is `agent` or `template` (fallback); `approval` is `auto`, `held` or `approved`. The broadcast layer gets one cue per moment; personal fields are resolved per viewer in the fan app.

4. Replay package (engine → replay player)

Already implemented in `engine/replay.py`: `frames`, `ball`, `moments` with `reveal: always | during | after`, `slow` window, and `status` text per audience (`stadium`, `fan`, `analyst`, `officials`).

Agents

Five agents each own one job and never compute a number; they read facts through the MCP server and hand off through a shared moment log, which is what the multi-agent judges score.

Agent	Job	Reads (MCP tools)	Writes	Fallback when it fails
Moment detector	Rank each event's significance; decide what gets airtime	`get_recent_events`, `get_rolling_metrics`	Moment entries with a score and a type	Rule-based thresholds (goal, card, VAR, speed over 30 km/h, pressure swing)
Explainer	Say why a moment matters: control vs. chaos, pressure shifts, rhythm	`get_moment`, `get_rolling_metrics`, `get_player_stats`	Fact-cited explanation per moment	Template sentence from the moment type
Decision Desk	Explain VAR checks and calls: what is checked, which law, why it is slow, then the evidence after the decision	`get_incident`, `get_law_summary`, `get_similar_incidents`	Incident explanation per phase	Replay templates already in `engine/replay.py`

Agent	Job	Reads (MCP tools)	Writes	Fallback when it fails
Persona	Rewrite one explanation for casual, analyst or ref-nerd mode, favorite club and language	Explanation + viewer profile	Per-audience text	Untranslated base text in the viewer's mode
Recap	Half-time and full-time story from the moment log	`get_moment_log`, `get_incidents`	Recap with three to five beats	Bullet list of the top moments by score

Orchestration: the orchestrator is Python code, not an agent. It subscribes to engine output, calls agents in order (detector → explainer or Decision Desk → persona), passes each output through the grounding validator, and emits overlay cues. Recap runs on `period_end`.

Grounding validator (deterministic): extracts every number, percentage, clock and event ID from agent text and checks each against the fact packet the agent was given. A miss triggers one retry with the violation named; a second miss uses the template. During a VAR check it also rejects any text containing a margin or verdict.

MCP server (`mcp_server/`): exposes match state as read-only tools: `get_recent_events`, `get_moment`, `get_moment_log`, `get_incident`, `get_incidents`, `get_rolling_metrics`, `get_player_stats`, `get_similar_incidents`, `get_law_summary`. Law summaries are short paraphrases written by you, never quoted IFAB text.

Model choice: a small, fast chat model deployment on Foundry for detector, persona and translation; a stronger model only for explainer, Decision Desk and recap. Cache every agent output by fact-packet hash so the demo replay costs nothing to rerun.

Tech stack and libraries

Python 3.12 for everything server-side, because Microsoft Agent Framework and the Foundry SDK are Python-first; React with TypeScript for the two web apps. Pin the exact versions you install on day one in `pyproject.toml` and `package-lock.json`, and don't upgrade mid-build.

Backend and agents

Package	Use	Version to start from
agent-framework	Agents, workflows (sequential, concurrent, handoff), MCP tool calls, checkpointing, human-in-the-loop	1.22.0 (Oct 15, 2026); `agent-framework-core` + `agent-framework-foundry` if you want a slimmer install
azure-ai-projects	Foundry project client, model deployments, evaluations	2.8.0
mcp	MCP server for match state (FastMCP, streamable HTTP)	2.x is current; pin `mcp<2` (1.30.0) if Agent Framework's MCP client needs 1.x
fastapi, uvicorn	Orchestrator API, producer console API, health checks	latest stable
pydantic v2	All four data contracts	latest stable
azure-eventhub	Simulator producer and engine consumer	latest stable
azure-messaging-webpubsubservice	Push overlay cues to browsers from Python	latest stable
azure-cosmos	Match state, incidents, moment log, audit log, agent output cache	latest stable
azure-identity	Managed identity in Azure, `AzureCliCredential` locally	latest stable
azure-cognitiveservices-speech	Optional voiced commentary (stretch)	latest stable

Package	Use	Version to start from
azure-monitor-opentelemetry	Traces and latency metrics into Application Insights	latest stable
pytest, httpx, ruff	Tests, API tests, lint and format	latest stable

Azure Web PubSub replaces SignalR from the earlier plan: it has a Python server SDK, while SignalR expects an ASP.NET or serverless backend.

Frontend (fan app, producer console)

Package	Use
React 18 + Vite + TypeScript	Both apps, one Vite workspace
@azure/web-pubsub-client	Receive overlay cues live
Plain SVG + `requestAnimationFrame`	Pitch, overlays and replays (already working in `web/replay.html`); no chart or animation library needed
zod	Validate incoming cues against the generated contract types
Vitest + Playwright	Component tests and one end-to-end replay test

Assets

- **Fonts:** Barlow Condensed (display), IBM Plex Sans (body), IBM Plex Mono (numbers), all from Google Fonts under the SIL Open Font License.
- **Icons:** Lucide (ISC license), only where a control needs one.
- **Club identity:** two invented crests drawn as simple SVG shields in kit colours: Northbridge amber `#F2B33D`, Ashvale blue `#5AA9E6`. No real club, league or broadcaster marks anywhere.
- **Video:** screen recordings only; royalty-free or no music; voiceover recorded yourself.

Developer tools that also score

GitHub Copilot agent mode while coding, the Azure Developer CLI (`azd`) and Bicep for deployment, and the Azure MCP server in VS Code for resource checks. Say which ones you used in the README, since the judges list them as hero technologies.

Repo structure and engineering practices

One public monorepo, one Python package per deployable service, and every rule from the requirements table backed by a test that runs in CI.

```
touchline/
  contracts/        Pydantic models for event, incident, cue, replay;
generates web/src/contracts.ts
  sim/              simulator, scenarios/*.json, Event Hubs producer
(exists)
  engine/           stats, offside, decision state machine, replays
(offside + replay exist)
  mcp_server/       FastMCP server exposing match state as read-only tools
  agents/           detector, explainer, decision_desk, persona, recap,
prompts/
  orchestrator/     FastAPI app: consumes engine output, runs agents,
validator, emits cues
  console/          producer console API (approve, hold, kill, audit log)
  evals/            harness, golden match, metrics report
  web/
    fan/            fan app: live overlays, personas, club, focus, pin,
language
    console/        producer console UI
    replay/         call replay player (exists as replay.html)
  infra/            main.bicep, modules/*.bicep, azure.yaml for azd
  tests/            unit and contract tests
  docs/             architecture diagram, demo script
  README.md
```

Practices that the judges score as "quality software development":

- **Contracts first:** a change to a contract fails the TypeScript build until the generated types are regenerated.
- **Tests:** unit tests for every engine function, contract tests for every cue, snapshot tests per persona, one Playwright test that plays the offside replay.

- **Eval harness (`evals/`):** replays the golden match with live agents and reports p50 and p95 latency, grounding first-try pass rate, fallback rate and premature-reveal violations. Commit the latest report as `evals/report.md` and link it in the README.
- **CI (GitHub Actions):** ruff, pytest, contract generation check, web build and Vitest on every push; the eval harness on demand, since it spends model credit.
- **Config:** no secrets in the repo; managed identity in Azure, `AzureCliCredential` locally, settings from environment variables.
- **Observability:** OpenTelemetry spans per event through every stage, so one trace shows simulator → engine → agent → validator → cue. Screenshot one trace for the video.
- **README:** one-paragraph pitch, architecture diagram, the Microsoft technologies used, how to run locally in one command, the eval report, and the link to the live demo.

Azure deployment and cost

Everything except model calls can run on free tiers or near-zero consumption billing, so the $100 student credit goes almost entirely to Foundry tokens; deploy all of it with one `azd up` from `infra/`.

Service	Tier	What it runs	Limit that matters
Microsoft Foundry	Pay per token	Model deployments for the agents	Check model quota on your student subscription on day one
Event Hubs	Basic, 1 throughput unit	Simulator → engine stream	1 consumer group, 1-day retention, 256 KB per event: only the engine reads the hub
Container Apps	Consumption, scale to zero	Engine, MCP server, orchestrator, console API	Monthly free grant of 180,000 vCPU-seconds, 360,000 GiB-seconds and 2 million requests
Web PubSub	Free	Live cues to browsers	20 concurrent connections, 20,000 messages a day

Service	Tier	What it runs	Limit that matters
Cosmos DB	Free tier, provisioned	Incidents, moment log, audit log, agent cache	1,000 RU/s and 25 GB; opt in at creation, not available on serverless
Static Web Apps	Free	Fan app, console UI, replay player	None that matters here
Application Insights	Pay as you go	Traces and latency metrics	Keep sampling on; small at this volume

Two decisions that keep the demo alive through judging:

- **Cached demo mode:** the fan app can play a recorded cue file of the golden match straight from Static Web Apps, with no Web PubSub, model or container calls. Make it the default link for judges, with a "Run live" button beside it. This covers the Web PubSub message cap and protects the credit.
- **Output cache:** every agent output is stored in Cosmos keyed by the hash of its fact packet, so a rerun of the same match never calls a model twice.

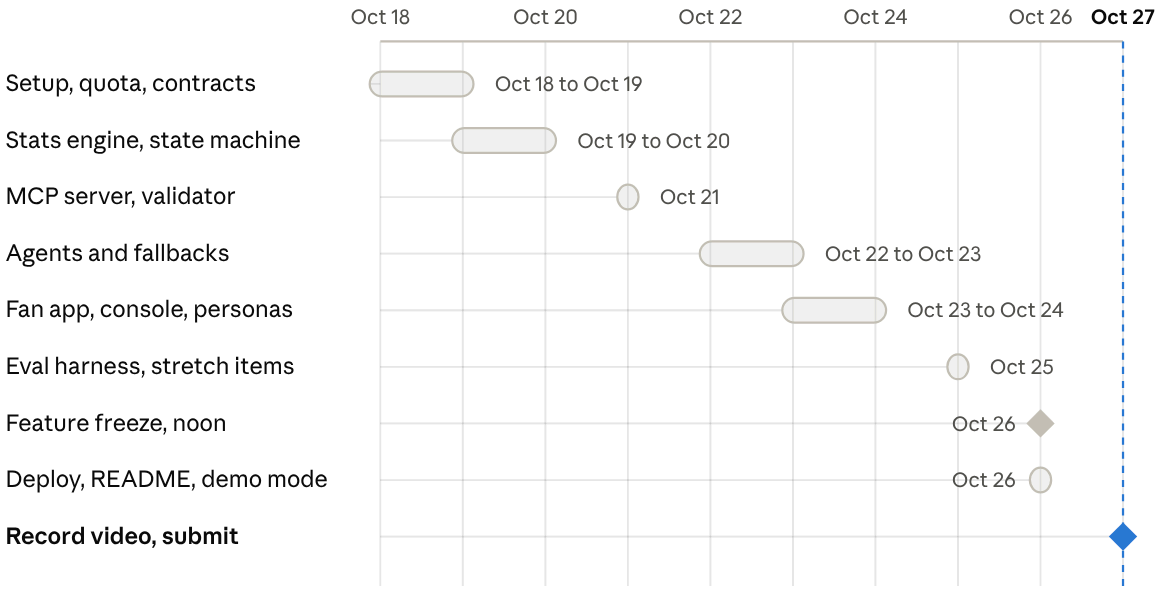
Cost controls on day one: a budget alert at $25, $50 and $75; `minReplicas: 0` on every container app; the smallest model that passes the eval for each agent; and `azd down` after Nov 10.

Build plan and milestones

Nine days of build with a hard feature freeze at noon on Oct 26, so the last day and a half go to deployment, the README and the video, never to new features.

Features freeze at noon Oct 26; submit on Oct 27

Build plan by day, Oct 18 to Oct 27, 2026



build plan · 9 workstreams, Oct 18 to 27

Each day ends with its exit check passing in CI; a day that slips takes time from stretch items first, never from the freeze.

Exit checks:

- [ ] Oct 19: repo restructured into the layout above, contracts generate TypeScript, Foundry model deployed and answering, budget alerts set
- [ ] Oct 20: pass difficulty, rolling control and pressure, and the decision state machine tested against the golden match
- [ ] Oct 21: an agent calls `get_incident` over MCP; the validator rejects a planted wrong number
- [ ] Oct 23: a full match runs through all five agents with templates on failure; nothing aired fails the validator
- [ ] Oct 24: Casual and Analyst screens differ; club, focus, pin and Spanish work live; console can hold and kill a cue
- [ ] Oct 25: eval report generated with all four non-functional targets met
- [ ] Oct 26: `azd up` from a clean clone works; cached demo link works on a phone
- [ ] Oct 27: video uploaded, submission sent by the afternoon

Demo video and submission checklist

The video must run under two minutes, because judges are not required to watch past that, and it should prove the trust thesis rather than tour features.

Time	Shot	What it proves
0:00–0:12	A VAR check with only "Check in progress" on screen	The problem: dead air and no explanation
0:12–0:45	Live match in the fan app: speed tag, pass difficulty, the 64′ pressure explainer, the VAR check, the decision with lines, the recap	The pipeline works end to end
0:45–1:05	Same decision for a Northbridge fan, an Ashvale fan, in Spanish, then Analyst mode	Personalization changes the experience, not just the words
1:05–1:20	The offside call replay with the stadium and officials′ views	Every call gets explained
1:20–1:40	Architecture diagram, then one trace: event → engine → agent → validator → cue	Agentic design and Azure integration
1:40–1:52	Kill the explainer agent live; the template takes over; the eval report on screen	Failure recovery and measured quality
1:52–2:00	"Built for the broadcaster's rendering path. The decision always stays with the officials."	Close

Submission checklist:

- [ ] Public GitHub repo with README, architecture diagram, eval report and license
- [ ] Elevator pitch naming the Microsoft and Azure technologies used and the problem solved
- [ ] Video under 2:00 on YouTube or Vimeo, public URL, no third-party marks or copyrighted music
- [ ] Live demo link that defaults to cached demo mode and works without login

- [ ] Testing instructions in English: how to open the demo, switch modes and run it locally
- [ ] Category chosen: Best Multi-Agent System
- [ ] Demo stays up and free until Nov 10, 2026
- [ ] Submitted by the afternoon of Oct 27, with the Projects page fields double-checked

Risks and guardrails

The biggest risk is not a bug but scope: every risk below has a fallback that keeps the demo working, and the feature freeze at noon on Oct 26 is the guardrail for all of them.

Risk	Early sign	Fallback
No model quota on the student subscription	Deployment fails on day one	Request quota at once; meanwhile build against a local model through Agent Framework's Ollama connector
Agent Framework or MCP version mismatch	MCP client errors on first integration	Pin `mcp<2`; if still broken, call the engine's functions as plain Agent Framework tools and keep the MCP server as a separate demo
Agent text too slow for live play	p95 over 2.5 s in the eval	Deterministic overlays go out first, agent text replaces the caption when ready; smaller model for that agent
Hallucinated numbers	Validator first-try pass rate under 90%	Tighter fact packets, one retry, then template; never air unvalidated text
Web PubSub message cap hit during judging	Messages near 20,000 a day	Cached demo mode is the default link
Credit runs out	Budget alert at $50	Freeze live mode, serve cached mode only
Simulated football looks unrealistic	Pass counts or speeds outside real ranges	The realism script already checks completion, distance and top speed; tune before adding features

Risk	Early sign	Fallback
Appearing to grade referees	Any copy that says a call was right or wrong	Officials' briefing states facts and method only; consistency ledger reports patterns, never verdicts
Rules and IP	Real names, marks or footage creep in	Fictional clubs only; no Premier League marks; laws paraphrased; music royalty-free or none

Sources

- Microsoft Premier League Hackathon official rules
- Microsoft Agent Framework 1.0 announcement
- agent-framework on PyPI
- azure-ai-projects on PyPI
- mcp on PyPI
- Azure Container Apps pricing
- Azure Web PubSub pricing
- Azure Cosmos DB free tier
- Event Hubs tier comparison